

ORDI: Orbit-Aware Redundant Distributed Inference for LEO Earth Observation Constellations

An-Che Liang, Guan-Ming Chiu, Pei-Yu Wu
National Taiwan University

Abstract—

Keywords—LEO constellations, Earth observation, distributed inference, fault-tolerant scheduling

II. Background and Related Work

Orbital edge computing treats a satellite constellation as a distributed computing platform rather than a collection of sensors that merely forward raw data. Denby and Lucia establish this design space and expose the coupled constraints of orbital motion, onboard compute, energy, sensing, and downlink [1]. Subsequent systems provide complementary mechanisms: Krios places and migrates persistent services across moving LEO resources [2], LEOEdge coordinates heterogeneous satellite and ground resources for AI inference [3], and SECO jointly schedules wide-area observation, routing, and compute placement [4]. These systems primarily target service availability, latency, coverage, or resource efficiency under nominal mobility.

Deadline-aware mobile-edge schedulers provide useful methods for dynamic arrivals and constrained communication. Liu et al. minimize the deadline violation ratio for dependent tasks through scheduling, migration, and merging [5], while TODG makes distributed online offloading decisions under hard delay constraints and stochastic communication resources [6]. These formulations motivate deadline misses, rather than average latency alone, as a reliability metric.

Selective replication is a separate approach to reducing delay and execution risk. Learning-based vehicular-edge schemes estimate worker quality and replicate tasks across a chosen set of workers [7]; their analysis further shows that excessive replication can increase queue contention and perform worse than a smaller replica count [8].

ORDI combines these three lines of work in an online scheduler for LEO Earth-observation inference. It splits requests into spatial tasks, places them on queue-aware helpers, and selects contact-aware routes that can complete computation and ground delivery before an absolute deadline. It also learns failure-domain risk from execution outcomes and selectively adds fault-disjoint backups when their expected reliability gain justifies the additional compute and communication cost. This integration makes deadline feasibility, orbital resource variation, and correlated failures part of one scheduling decision.

III. Modeling

A. System and Task Model

Orbital contacts, compute queues, and energy-thermal states evolve over time, whereas continuously recomputing the schedule after every state change would incur prohibitive control and optimization overhead. We therefore adopt a rolling-horizon discretization in which scheduling decisions are revised at fixed epochs indexed by $\ell \in \mathbb{Z}_{\geq 0}$. Each epoch has duration Δ and interval

$$\mathcal{I}_\ell = [t_\ell, t_{\ell+1}), \quad t_\ell = \ell\Delta. \quad (1)$$

At t_ℓ , the scheduler refreshes the satellite states and pending task set, and constructs a rolling-horizon plan using contacts predicted through the relevant task deadlines. The epoch length determines the replanning cadence; it does not restrict an operation to the current interval, and resource reservations may extend beyond $\mathcal{I}_\ell$. Hereafter, t denotes the current decision time t_ℓ .

The constellation is represented by a time-varying directed contact graph $\mathcal{G}(t) = (\mathcal{N}, \mathcal{E}(t))$, where $\mathcal{N} = \mathcal{S} \cup \mathcal{D}$ contains satellites $\mathcal{S}$ and ground stations $\mathcal{D}$. A contact $e = (i, j, o_e, c_e, R_e, \rho_e, \kappa_e)$ denotes a directed communication opportunity from i to j , where o_e and c_e are its opening and closing times, respectively. During the interval $[o_e, c_e]$, the contact has rate R_e , success probability ρ_e , and type $\kappa_e \in \{\text{ISL}, \text{downlink}\}$. Its raw capacity is $K_e = R_e(c_e - o_e)$ bits. A feasible store-and-forward route is a time-ordered sequence of contacts and may include waiting at intermediate satellites.

At decision time t , satellite i is characterized by

$$\sigma_i(t) = (C_i(t), B_i(t), \Theta_i(t), Q_i(t), A_i(t), \rho_i(t)), \quad (2)$$

where C_i is its effective compute rate in FLOP/s, B_i is battery energy, Θ_i is payload temperature, Q_i is already-queued work in FLOPs, $A_i \in \{0, 1\}$ is availability, and ρ_i is node reliability. The effective rate C_i incorporates thermal throttling. Let $H_i(t)$ indicate whether the platform is operational. Availability is defined as

$$A_i(t) = \mathbb{1}[H_i(t) = 1, B_i(t) \geq B_i^{\min}, \Theta_i(t) < \Theta_i^{\max}]. \quad (3)$$

The battery and thermal dynamics update C_i and A_i between consecutive decision epochs.

Each EO request k is represented as $\mathcal{T}_k = (s_k, r_k, D_k, \mathcal{V}_k)$, where s_k is the imaging satellite, r_k is the release time, D_k is the absolute delivery deadline, and

$\mathcal{V}_k$ is a grid of image tiles. Each tile $v \in \mathcal{V}_k$ is described by

$$\tau_{kv} = (d_{kv}^{\text{in}}, d_{kv}^{\text{out}}, w_{kv}), \quad (4)$$

where d_{kv}^{in} and d_{kv}^{out} are input and result sizes in bits, and w_{kv} is inference work in FLOPs. The two data volumes are distinguished because onboard inference transforms a high-dimensional image tile into a task-dependent product, such as a class label, bounding boxes, or a segmentation mask.

Orbital motion is predictable; therefore each satellite is assumed to know the contact graph over the scheduling horizon. This knowledge specifies when links will exist, but not the contemporaneous resource state of another satellite. At time t , satellite i directly observes only $\sigma_i(t)$. For $j \neq i$, its most recent estimate is

$$\hat{\sigma}_j^i(t) = \sigma_j(t_j^i), \quad a_j^i(t) = t - t_j^i, \quad (5)$$

where t_j^i is the generation time of the latest state advertisement received by i , and a_j^i is its age. The state of j is unknown to i until such an advertisement is received.

Neighboring satellites exchange state advertisements and task data over inter-satellite links. To limit flooding overhead and preserve contact capacity for task traffic, each epoch's state advertisement is forwarded over at most three ISL hops. Let $\mathcal{A}_i^{(h)}(e)$ be the advertisements available to satellite i after at most h forwarding hops in epoch e , and let $\mathcal{N}_i(e)$ be its incoming ISL neighbors during that epoch. The exchange can be summarized as

$$\begin{aligned} \mathcal{A}_i^{(0)}(e) &= \{(i, \sigma_i(t_e), t_e)\}, \\ \mathcal{A}_i^{(h)}(e) &= \text{fresh} \left(\mathcal{A}_i^{(h-1)}(e) \cup \bigcup_{j \in \mathcal{N}_i(e)} \mathcal{A}_j^{(h-1)}(e) \right), \\ h &= 1, 2, 3. \end{aligned} \quad (6)$$

where $\text{fresh}(\cdot)$ retains only the newest advertisement per origin. A received entry becomes usable only after its contact-constrained arrival time. This bounded dissemination extends state awareness beyond immediate neighbors without requiring network-wide flooding. Task data may traverse multiple ISLs, and a downlink delivers the resulting data to a ground station. Control and task traffic share the finite capacities of the corresponding contact windows. Thus, knowledge of future connectivity is global and deterministic, whereas knowledge of remote satellite state is local, delayed, and acquired through communication.

B. Constraints and Problem Formulation

For each tile $v \in \mathcal{V}_k$, the decisions include a split width $q_{kv} \in \{1, 2, 4\}$ and a reconstruction-group count $n_{kv} \in \{1, 2\}$. Following similar distributed-inference work [9], [10], we allow at most four shards because over-sharding increases communication, synchronization, and scheduling overhead.

A reconstruction group is an end-to-end plan that independently completes tile v , indexed by $g \in \{1, \dots, n_{kv}\}$.

We choose $n_{kv} \leq 2$ because each additional group schedules another q_{kv} shard placements, consuming compute and link capacity while enlarging the combinatorial placement search.

For group g , shard $m \in \{1, \dots, q_{kv}\}$ carries $d_{kv}^{\text{in}}/q_{kv}$ input bits, requires w_{kv}/q_{kv} FLOPs, and produces $d_{kv}^{\text{out}}/q_{kv}$ result bits. Its placement

$$x_{kvgm} = (h_{kvgm}, P_{kvgm}^{\text{in}}, P_{kvgm}^{\text{gnd}}) \quad (7)$$

where h_{kvgm} is the compute satellite, P_{kvgm}^{in} and P_{kvgm}^{gnd} are the time-respecting input and output paths, respectively. Each path contains at most $H_{\text{max}}^{\text{ISL}} = 12$ ISL hops.

Let F_{kvgm}^{in} be the time at which the input shard reaches its compute satellite, and let S_{kvgm}^{cmp} be its computation start times. The shard computation is constrained by

$$\begin{aligned} S_{kvgm}^{\text{cmp}} &\geq F_{kvgm}^{\text{in}}, \\ \int_t^{S_{kvgm}^{\text{cmp}}} C_{h_{kvgm}}(\tau) d\tau &\geq Q_{h_{kvgm}}(t), \\ \int_{S_{kvgm}^{\text{cmp}}}^{F_{kvgm}^{\text{out}}} C_{h_{kvgm}}(\tau) d\tau &= \frac{w_{kv}}{q_{kv}}. \end{aligned} \quad (8)$$

The result path starts after computation. Let F_{kvgm} denote the time at which the result reaches the ground. A group completes after all q_{kv} of its results arrive, while a tile completes when any selected reconstruction group completes:

$$T_{kv} = \max_{1 \leq m \leq q_{kv}} F_{kvgm}, \quad T_{kv} = \min_{1 \leq g \leq n_{kv}} T_{kv}^g. \quad (9)$$

Every admitted placement obeys

$$\begin{aligned} r_k &\leq t, \quad F_{kvgm} \leq D_k, \\ A_i(t) &= 1, \quad \forall i \in \mathcal{S}(P_{kv}^{\text{in}} \cup P_{kv}^{\text{gnd}}). \end{aligned} \quad (10)$$

Here, $\mathcal{S}(P)$ denotes the satellite nodes traversed by path P . At decision time, a satellite evaluates remote availability using its latest estimate from Equation (5); stale information can therefore cause a planned placement to fail during execution.

Shared resources are explicitly serialized. For every contact window e ,

$$\sum_{x_{kvgm} \ni e} b_{kvgm,e} \leq K_e, \quad (11)$$

and each transmission interval must lie inside $[o_e, c_e]$. Intervals using the same satellite terminal cannot overlap, and compute intervals on one compute satellite cannot overlap. State advertisements and task transfers consume capacity from the same contact-window budget.

Within a group, compute satellites are distinct. A backup group must additionally avoid the primary compute satellites, their orbital planes, and satellite nodes already used by the primary routes. If $\delta(i)$ denotes the fault domain of satellite i , then for primary group 1 and each selected backup group $2 \leq g \leq n_{kv}$,

$$\delta(h_{kv1m}) \neq \delta(h_{kvgm'}) \quad \forall m, m' \in \{1, \dots, q_{kv}\}, \quad (12)$$

with the analogous node-disjoint condition on their routes.

Let $\mathcal{C}_{kvg}$ be the physical node and link components required by group g . Its success probability is $p_{kvg} = \prod_{c \in \mathcal{C}_{kvg}} \rho_c$. Because groups can share a source or contact component, tile reliability uses inclusion–exclusion rather than assuming independent replicas:

$$P_{kv} = \sum_{\emptyset \neq H \subseteq \{1, \dots, n_{kv}\}} (-1)^{|H|+1} \prod_{c \in \bigcup_{g \in H} \mathcal{C}_{kvg}} \rho_c, \quad (13)$$

Let $L_{kv} = T_{kv} - t$ be the service latency, and let Z_{kv} be the total routed bits counted per hop. The online scheduling problem is

$$\max_{\{q_{kv}, n_{kv}, x\}} \sum_k \sum_{v \in \mathcal{V}_k} [P_{kv} e^{-\alpha L_{kv}} - \lambda_C Z_{kv} - \lambda_R (n_{kv} - 1)] \quad (14)$$

subject to Equations (8)–(12), with $q_{kv} \in \{1, 2, 4\}$ and $n_{kv} \in \{1, 2\}$. The physical model separately integrates compute and radio energy and updates B_i and Θ_i ; therefore energy affects subsequent feasibility through Equation (3) rather than through an optimistic per-placement estimate.

References

- [1] B. Denby and B. Lucia, “Orbital edge computing: Nanosatellite constellations as a new class of computer system,” in Proc. ACM ASPLOS, 2020.
- [2] V. Bhosale, A. Gavrilovska, and K. Bhardwaj, “Krios: Scheduling abstractions and mechanisms for enabling a LEO compute cloud,” in Proc. ACM SoCC, 2024.
- [3] J. Su, Y. Liu et al., “LEOEdge: A satellite-ground cooperation platform for AI inference in large LEO constellation,” IEEE Journal on Selected Areas in Communications, vol. 43, no. 1, pp. 36–51, 2025.
- [4] Z. Zhai, L. Zeng, T. Ouyang, S. Yu, Q. Huang, and X. Chen, “SECO: Multi-satellite edge computing enabled wide-area and real-time earth observation missions,” in Proc. IEEE INFOCOM, 2024.
- [5] S. Liu et al., “Dependent task scheduling and offloading for minimizing deadline violation ratio in mobile edge computing networks,” IEEE Journal on Selected Areas in Communications, 2023.
- [6] S. Yue, J. Ren, N. Qiao, Y. Zhang, H. Jiang, Y. Zhang, and Y. Yang, “TODG: Distributed task offloading with delay guarantees for edge computing,” IEEE Transactions on Parallel and Distributed Systems, 2022.
- [7] Y. Sun, J. Song, S. Zhou, X. Guo, and Z. Niu, “Task replication for vehicular edge computing: A combinatorial multi-armed bandit approach,” in Proc. IEEE GLOBECOM, 2018. [Online]. Available: <https://arxiv.org/abs/1807.05718>
- [8] Y. Sun, S. Zhou, and Z. Niu, “Distributed task replication for vehicular edge computing: Performance analysis and learning-based algorithm,” IEEE Transactions on Wireless Communications, 2021. [Online]. Available: <https://arxiv.org/abs/2002.08833>
- [9] Z. Zhao, K. M. Barijough, and A. Gerstlauer, “DeepThings: Distributed adaptive deep learning inference on resource-constrained IoT edge clusters,” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 37, no. 11, pp. 2348–2359, 2018.
- [10] E. Tang, X. Guo, and T. Stefanov, “The effects of partitioning strategies on energy consumption in distributed CNN inference at the edge,” arXiv preprint arXiv:2210.08392, 2022.